

CNN vs. ResNet18: Deep Learning Image Classification on STL-10

Jack Gauderman¹, Abel Mendoza¹, Emily Nguyen², Samuel Wade¹

¹Department of Electrical and Computer Engineering, Texas A&M University, College Station, TX, USA

²Department of Statistics, Texas A&M University, College Station, TX, USA

jackg@tamu.edu, abelm98@tamu.edu, nmkn28@tamu.edu, samuelwade@tamu.edu

Abstract—This paper aims to evaluate convolutional neural networks (CNN) specifically targeted to multi-class image classification purposes utilizing STL-10 datasets. The STL-10 dataset contains 10 different classes consisting of 500 training images and 800 testing images for each class at 96×96 resolution. We created a convolutional neural network (CNN) using PyTorch, incorporating data augmentation, hyperparameter tuning, and comprehensive evaluation metrics. In addition to the CNN, we fine-tuned and evaluated a pretrained ResNet18 model to compare the CNN’s architecture to a deeper model. Experimental results show that CNN achieves good performance, while ResNet18 improves classification accuracy over complex classes.

Index Terms—Convolutional Neural Networks, ResNet18, Image Classification, Deep Learning, STL-10, Computer Vision

I. INTRODUCTION

Image classification is integral to many computer vision-related applications such as autonomous vehicles, retail, security, medical imaging, and defense. Deep convolutional neural networks (CNNs) have transformed this field by ingesting and applying multi-level representations of the raw pixels instead of depending on hand-crafted features [1].

The STL-10 dataset represents a difficult challenge that deals with higher resolution (96×96 pixels) compared to CIFAR-10 (32×32) but limited labeled data with only 500 training images per class [2]. This constraint makes STL-10 more appropriate for observing generalization with limited supervision, replicating real-world situations where annotation is an expensive operation. The dataset includes 10 balanced classes: airplane, bird, car, cat, deer, dog, horse, monkey, ship, and truck.

Our deliverables involve implementing a custom CNN architecture and fine-tuning ResNet18 on the dataset, applying appropriate data preprocessing and augmentation, conducting systematic hyperparameter tuning, conducting comprehensive performance evaluation with multiple metrics, and analyzing model interpretability.

II. RELATED WORK

A. STL-10 Dataset

Coates et al. [2] introduced STL-10 for the application of developing fully autonomous learning algorithms capable of being unsupervised. It is inspired by the CIFAR-10 dataset, but is more difficult due to higher resolution images (96×96 compared to 32×32), and 10 times less training images per

class. STL-10 also includes 100,000 unlabeled images from a broader distribution of classes meant for unsupervised learning, but we won’t be using these in our report. All images in the STL-10 dataset were taken from labeled images in the ImageNet dataset.

B. CNN Architecture

Convolutional neural networks (CNNs) were first successfully demonstrated by LeCun et al. [3] specifically tailored to document recognition. This particular use case established the conceptual foundation for CNNs that are still in use today, although modern architectures have evolved significantly. The three main parts that make up your standard CNN include convolutional, fully connected, and pooling layers. Convolutional layers help apply filters typically taking up form factors of 3×3 or 5×5 that navigate through the inputted image to highlight features such as edges, shapes, and textures. Each specific filter takes part in making a feature map that shows where certain patterns are located in the image. Pooling layers, such as max pooling, help decrease the number of spatial dimensions by only accepting the highest value from small areas. This makes the process translation-invariant and less expensive to compute. Finally, all the layers together produce class predictions given the combined extracted features.

C. ResNet18 Architecture

ResNet18 is an 18-layer deep residual network to optimize and allow deeper networks introduced by He et al. [1]. Its key innovation is the use of “skip connections” that facilitate the training of deeper networks by overcoming the vanishing gradient problem present in CNN architecture. This is accomplished by providing a direct path that skips over one or more layers and simply copies the input to the output of the residual block unaltered.

It is constructed with 18 total layers consisting of an initial 7×7 convolution and max-pooling operation, then four stages of residual blocks (eight blocks in total). Each block contains two 3×3 convolutional layers and. The model ends with a final fully connected layer. Overall, this architecture provides an easier learning curve and enables easier optimization for deeper neural networks.

III. METHOD

A. Data Augmentation

All STL-10 images were normalized using the mean and standard deviation of the training dataset, and converted into tensors, which are required for PyTorch model inputs.

Many relatively aggressive additional data augmentations are applied to the training dataset. The goal is to reduce overfitting and enhance model robustness and generalization. This is especially important on a smaller dataset like STL-10, which can be prone to overfitting without augmentations. All of these augmentations help the model to learn more diverse features without collecting more images.

Random resized crop: Each image is randomly cropped and resized to 96x96 pixels, with a scale between 50% to 100% of the original image size. Random resized crop introduces several spatial variations and forces the model to learn more features at different scales and locations.

Random horizontal flip: Each image has a 50% probability of being horizontally flipped. As same as random resized crop augmentation, horizontal flipping also forces the model to learn more diverse features.

Random augmentation: Two augmentations are randomly selected with a magnitude of 5 without manual augmentation selections. The selected augmentations could be brightness, contrast, and saturation adjustments, rotations, shearing and shifting transformations, color distortions, and other image augmentation methods.

Random erasing: A random part of the image has a 20% probability of being erased. The randomly erased area is between 2% to 20% of each image. Random erasing forces the model to learn different portions of the image rather than focus on just one specific area.

B. Data Splitting

Next, we had to split our data to create a validation split, because STL-10 only has train and test splits by default. We randomly moved 20% of the training split into a separate validation split, and left the test data untouched. Our final distribution for our sets were 4000 training images, 1000 validation images, and 8000 test images across all 10 classes.

C. Exploratory Data Analysis

Before looking into the class distribution, we output three sample images for each class from the STL-10 dataset. These images are representative of the 10 different classes, giving an idea of how the dataset looks.

We then examined the class distributions within the training, testing, and validation sets to identify potential class imbalance. Figure 2 shows that STL-10 has well-balanced data across three sets, suggesting there would be no model biases due to class imbalance.

To further visualize our data, we apply t-SNE dimensionality reduction to view a graph of our training dataset in 2 dimensions. Because t-SNE is computationally expensive, we first apply PCA to reduce images from 27,648 features down to 100, and then reduce down to 2 features with t-SNE to capture



Fig. 1. Sample Images

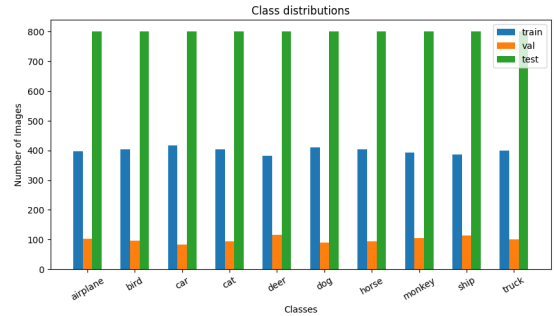


Fig. 2. Class distribution in the training, validation, and test splits

the most complex data possible. Figure 3 shows a meaningful structure in the data. Vehicle (airplane, ship, truck, car) and animal (bird, horse, cat, deer, dog, monkey) classes were separated. However, there is still significant overlap within animal subclasses.

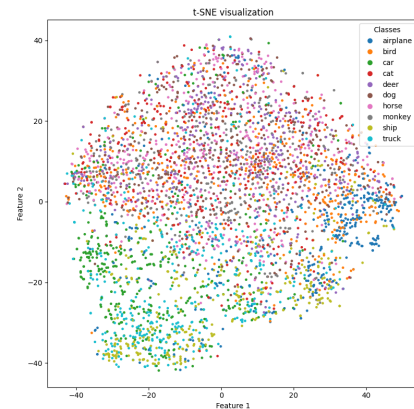


Fig. 3. t-SNE visualization of PCA-reduced STL-10 features

D. Model Architecture

Neural networks are the best choice for image classification because of their capability to capture highly nonlinear relationships in extremely high-dimensional data. Compared to traditional algorithms such as KNN and decision trees, neural

networks can achieve significantly higher performance without extensive feature engineering. For this report, we first look at a basic CNN that we developed ourselves, then look at ResNet18 pretrained on ImageNet to maximize classifier performance.

a) CNN: We first used a convolutional neural network (CNN) because of the STL-10 spatial structure of images. In the STL-10 dataset, each 96x96 RGB image contains over 27,000 pixels. Fully connected neural networks would treat each of the images as an independent feature, ignoring the spatial relationships between pixels. As a result, it requires a massive number of parameters for data training. CNNs address this issue by using convolutional filters and pooling layers that capture the similarity of nearby pixels while drastically reducing the size of parameters.

Our CNN model follows a standard structure of CNN structure. A convolutional backbone progressively reduces spatial resolution and increases channel depth. A compact classification head then maps the result to class logits.

Backbone The backbone consists of five sequential convolutional blocks, used mainly for CNN feature extraction. Each block applies a convolution of kernel size 3 with padding 1 to scan the image in the 3x3 window and preserve the spatial image size, followed by batch normalization to reduce overfitting and stabilize the training process, and ReLU activation for nonlinear feature extraction to learn complex image patterns and avoid gradient vanishing. After each block, a 2x2 max-pooling layer is applied to gradually reduce the input images from 96x96 pixels to 48x48, 24x24, 12x12, 6x6, and finally 3x3. The number of channels goes deeper from 3 to 64, 128, 256, 512, and finally 1024, allowing the model to learn more complex and high-level patterns essential for more accurate classification.

Head The classification head of CNN takes the features extracted from the backbone and makes final predictions based on class logits. An adaptive average pooling layer compresses each 6x6 feature map into 1x1 representation. The pooled features are then flattened and passed through ReLU activation. The final linear layer outputs logits for 10 classes.

b) ResNet18: We then used a ResNet18 model, an advanced CNN model that incorporates residual blocks and skip connections to address vanishing gradient problems in deeper networks. ResNet18 contains 18 layers and is able to learn more complex features than a standard CNN. However, training ResNet18 from scratch on STL-10 was not ideal since the model is too large and easily overfits by memorizing the training images. Instead of training the entire network, we used a pretrained ResNet18, which had already been trained on the large ImageNet dataset. The model had learned general visual features, then it could immediately generalize well to our chosen dataset with additional fine-tuning.

IV. EXPERIMENT RESULTS AND DISCUSSION

A. Experimental setup

Our dataset was split into 4,000 training images, 1,000 validation images, and 8,000 testing images. Both CNN and ResNet18 models were evaluated on this dataset. Several

experimental setups were used for each combination of hyperparameters (learning rate and weight decay). For each hyperparameter setup, both models were trained using the same model architecture.

Training Both models were trained using Cross Entropy Loss with label smoothing of 0.1 as the objective function, which is appropriate for multi-class image classification problems. We chose the Adam optimizer due to its adaptive learning rate mechanism and its lower sensitivity to hyperparameters compared to other optimizers like SGD.

We experimented with the learning rate and weight decay values ranging from 1e-4 to 1e-3. The experiments were monitored using validation accuracy. We found the following hyperparameter setups provided a good balance between stable convergence and training speed and yielded the highest validation accuracy:

- CNN: learning rate = 1e-3, weight decay = 1e-3
- ResNet18: learning rate = 1e-4, weight decay = 1e-3

To reduce overfitting and ensure smoother convergence, we used a Cosine Annealing Learning Rate scheduler for both models. The batch size for all experiments was set to 128. CNN model was trained for 50 epochs, and ResNet was trained for 20 epochs.

Testing Model performance was evaluated by performing five independent experimental runs for both CNN and fine-tuned ResNet18. All runs used the same training configurations. For each run, we recorded the test accuracy, test loss, confusion matrices, and precision, recall, and f1-score for each class. Running multiple experiments allowed us to compute and report the mean and standard deviation for each metric for more stable model evaluation.

B. Results

a) CNN: The CNN model achieved a mean test accuracy of $76.28\% \pm 0.64\%$ with a mean test loss of 1.0855 ± 0.0097 over five experimental runs. This reflected moderate test accuracy but relatively high test loss. The model was able to learn meaningful visual features but still made incorrect predictions.

TABLE I
CNN CLASSIFICATION REPORT (MEAN $\pm$ STD OVER 5 RUNS)

Class	Precision	Recall	F1-score	Support
airplane	0.8747 ± 0.0076	0.8635 ± 0.0077	0.8690 ± 0.0051	800
bird	0.7401 ± 0.0138	0.7308 ± 0.0150	0.7353 ± 0.0134	800
car	0.9135 ± 0.0068	0.8705 ± 0.0132	0.8914 ± 0.0079	800
cat	0.6542 ± 0.0221	0.6685 ± 0.0158	0.6608 ± 0.0051	800
deer	0.7308 ± 0.0110	0.7137 ± 0.0141	0.7221 ± 0.0108	800
dog	0.5866 ± 0.0132	0.5755 ± 0.0168	0.5808 ± 0.0117	800
horse	0.7638 ± 0.0076	0.7730 ± 0.0153	0.7683 ± 0.0105	800
monkey	0.6905 ± 0.0051	0.7235 ± 0.0224	0.7064 ± 0.0096	800
ship	0.8566 ± 0.0091	0.8892 ± 0.0050	0.8726 ± 0.0063	800
truck	0.8265 ± 0.0078	0.8197 ± 0.0092	0.8230 ± 0.0049	800
Average	0.7637 ± 0.0061	0.7628 ± 0.0064	0.7630 ± 0.0063	8000

Vehicle classes Our CNN performed well on vehicle classes. All precision and recall values were above 0.82 (Table I). Vehicles' images typically have more clean and well-defined shapes, sharper edges, and less ambiguous patterns, making them easier to identify without the need of a complex model.

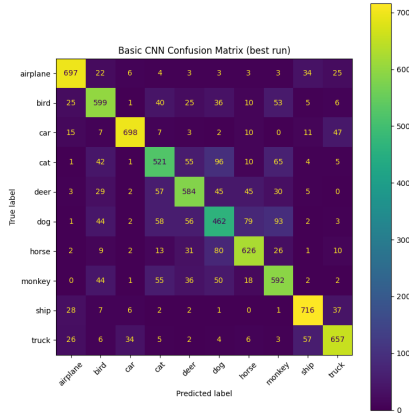


Fig. 4. Confusion matrix for the CNN on the STL-10 test set

The strongest classes were airplane and ship with 697 and 716 correct predictions, respectively. These are the two easiest classes with distinct shapes such as airplanes wings, sharp ship outlines, and consistent background of water or skies. They have fewer overlapping features with the other classes.

Trucks were primarily confused with cars (110 times, Figure 4). This was expected since cars and trucks have similar visuals, such as wheels, windshield, body shapes, and share the same backgrounds.

Animal classes CNN showed confusion across animal classes. Animal images often have several different textures, fur patterns, background, and poses that confuse CNN since the model was not able to distinguish high-level features and separate animals using subtle differences. Classes like cat and dog showed the poorest performance, with precision and recall around 0.65 and below. The model performed moderately on horse, bird, and deer classes, with precision and recall above 0.73 (Table I).

CNN performed decently on bird, deer, horse, and monkey, with 599, 571, 626, and 592 correct predictions, respectively (Figure 4). CNN was still able to separate animals with mid-level patterns. These classes benefit from their body shapes and less variability. For instance, birds have wings and a sky background. Monkeys are surrounded by trees. Horses have elongated body shapes. Deer have antlers. These are distinctive features that help CNN to separate. However, at a lower resolution, these classes lose their distinctive features.

Cats and dogs had the lowest predictions among all classes, aligning with the result from the classification report. Based on Figure 4, cat class performed slightly better with 521 correct classifications, but was constantly mistaken for dog (96 times), monkey (65 times), and deer (55 times). It was expected since they share similar sizes, fur textures, soft outlines, and background.

Dog class had the lowest correct prediction with only 463 correct classification with 93 times misclassified as monkeys, 80 times misclassified as horses, and 56 times misclassified as cats. Dogs have several sizes, and colors that overlap with other animals.

b) ResNet18: The ResNet18 model achieved a mean test accuracy of $87.84\% \pm 0.31\%$ with a mean test loss of 0.8291 ± 0.0065 over five experiments. ResNet18 showed significant improvement over the CNN model with higher test accuracy and lower test loss. The model was able to make more confident, correct predictions with a small standard deviation, suggesting training stability and consistency over five runs. ResNet18’s deeper architecture allowed it to learn more complex features and generalize better on the STL-10 dataset.

TABLE II
RESNET18 CLASSIFICATION REPORT (MEAN $\pm$ STD OVER 5 RUNS)

Class	Precision	Recall	F1-score	Support
airplane	0.9164 ± 0.0084	0.9192 ± 0.0042	0.9178 ± 0.0031	800
bird	0.9373 ± 0.0057	0.8965 ± 0.0065	0.9164 ± 0.0048	800
car	0.9382 ± 0.0065	0.9217 ± 0.0073	0.9299 ± 0.0028	800
cat	0.8225 ± 0.0211	0.7987 ± 0.0211	0.8100 ± 0.0070	800
deer	0.8499 ± 0.0143	0.8770 ± 0.0092	0.8631 ± 0.0082	800
dog	0.8061 ± 0.0112	0.7690 ± 0.0197	0.7869 ± 0.0098	800
horse	0.8540 ± 0.0173	0.8755 ± 0.0153	0.8644 ± 0.0077	800
monkey	0.8555 ± 0.0115	0.8830 ± 0.0146	0.8689 ± 0.0049	800
ship	0.9175 ± 0.0157	0.9485 ± 0.0098	0.9326 ± 0.0054	800
truck	0.8896 ± 0.0083	0.8953 ± 0.0099	0.8923 ± 0.0043	800
Average	0.8787 ± 0.0031	0.8784 ± 0.0031	0.8782 ± 0.0031	8000

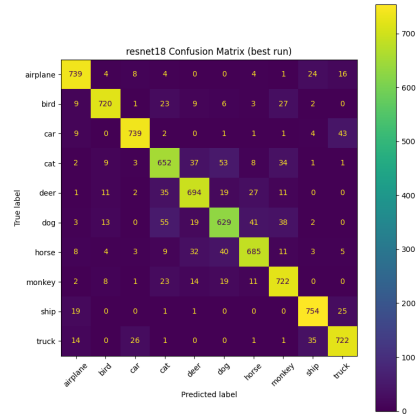


Fig. 5. Confusion matrix for ResNet18 on the STL-10 test set

Vehicle classes ResNet18 showed strong classification performance across all classes. Similar to CNN, the model achieved the highest precision and recall on the easier classes such as ship, airplane, car, and truck, which have less ambiguous patterns. However, unlike CNN, from Table II, ResNet18 also performed well on the animal classes, with all precision and recall scores above 0.80. This suggested the model’s ability to capture more complex texture, shape, and features that CNN failed to learn.

The strongest classes were still airplane and ship, with 739 and 754 correct predictions, respectively (Figure 5). These are still the easiest classes for the model. The deeper model architecture was able to separate images with sharp and well-defined outlines and was even more effective than the CNN.

ResNet18 still had some confusion between trucks and cars, but the number misclassification decreased to only 26, significantly reduced compared to CNN model (Figure 5).

ResNet18 was now able to separate vehicles with overlapping shapes and structures, thanks to the residual connections and deeper architectures.

Animals classes The model performed better on cat and dog classes that CNN struggled to separate. Precision and recall scores are approximately 0.80 and above (Table II). The deeper network architectures and residual connections allowed the model to handle complex and high-variability textures, shapes, and patterns in the STL-10 data.

The model performed the best on bird, deer, horse, and monkey with 720, 649, 685, and 722 correct predictions, respectively (Figure 5). ResNet18 separated these classes more effectively.

The improvement was noticeable in more confusing classes like cat and dog, which had the poorest performance under the CNN model. From Figure 5, cat class now had 652 correct predictions, and the misclassification into dogs was reduced to 56 (down from 110 in CNN model). Dog class also improved significantly with 629 correct predictions, and misclassification into other animal classes now reduced. ResNet18 successfully learned ambiguous features and other variations to separate animal classes.

C. Discussion

TABLE III
MODEL PERFORMANCE COMPARISON

Model	Test Accuracy (mean $\pm$ std)	Test Loss (mean $\pm$ std)
Basic CNN	76.28% $\pm$ 0.64%	1.0855 $\pm$ 0.0097
ResNet18	87.84% $\pm$ 0.31%	0.8291 $\pm$ 0.0065

The experimental results clearly showed that ResNet18 outperformed CNN across all metrics. Based on the comparison in Table III, ResNet18 achieved both higher mean test accuracy and lower mean test loss. Its mean test accuracy was 87.84% $\pm$ 0.31% and its mean test loss was 0.8291 $\pm$ 0.0065, compared to the CNN's mean test accuracy of 76.28% $\pm$ 0.64% and test loss of 1.0855 $\pm$ 0.0097. ResNet18's lower standard deviation for both metrics suggested higher confidence, consistency, and generalizability across the five experimental runs.

Both CNN and ResNet18 performed well on classes with distinctive features and easier separability. In the STL-10 dataset, the vehicle classes are easier due to clear structures. ResNet18 performed slightly better since it is able to classify these classes with higher accuracy and fewer misclassification, especially in commonly confused classes like trucks and cars.

However, two model performance differences appeared in the animal classes. While ResNet18 performed strongly over both vehicle and animal classes, CNN failed to effectively separate these categories. Animal classes contain more overlapping features and higher-level patterns that only deeper model architectures like ResNet18 were able to capture. CNN performed better on animal classes with more distinctive features such as birds, monkeys, deer, and horses. These animals have more distinguishing features and different backgrounds. In contrast, cats and dogs share similar features

and backgrounds, which caused CNN models to struggle and mistake them into similar classes. ResNet18 showed a clear improvement in separating these two classes. The number of misclassifications was greatly reduced.

Overall, CNN captured lower-level features, while ResNet18 extracted higher-level and complex patterns. ResNet18 leverages its deeper layers and residual connections to learn more subtle features, allowing it to perform better on STL-10 dataset.

V. CONCLUSION

In this paper, we successfully developed a CNN for multi-class image classification on the STL-10 dataset, and fine-tuned ResNet18 to boost performance even further. Throughout this project we learned that CNNs offer many advantages over traditional fully connected networks when working with images. Additionally, we learned the importance of data augmentation and hyperparameter tuning to reduce overfitting and boost performance. Finally, we learned how to analyze our results and confusion matrices to draw conclusions and interpret the output of a black box model.

Practical applications of image classification are numerous. Healthcare, defense, agriculture, and automotive industries all have obvious applications. In healthcare, image classification can be used for X-rays, MRI scans, cell classification and more. For defense, image classification can be used for target detection. In the agriculture and automotive sectors, image classification is necessary in the development of self-driving machinery. CNNs and resnet models are used across all of these areas and more to create autonomy, speed up processes, and save money.

VI. PROJECT WEBSITE

We took the extra credit option number 1 to make a project website. The website is available at the following link: https://emily-ng28.github.io/ecen758_finalproject/

REFERENCES

- [1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770-778.
- [2] A. Coates, H. Lee, and A. Y. Ng, "An Analysis of Single Layer Networks in Unsupervised Feature Learning," in Proc. 14th Int. Conf. Artificial Intelligence and Statistics (AISTATS), 2011, pp. 215-223.
- [3] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," Proc. IEEE, vol. 86, no. 11, pp. 2278-2324, Nov. 1998.